

README

Bibliografia

3D Models and textures

- Fab Quixel
- Modelos propios

Fuentes informativas que revisamos para este trabajo

Luces

<https://www.youtube.com/watch?v=ToCSRyXva5w>

<https://stackoverflow.com/questions/25259622/opengl-es-2-0-how-to-set-ight-sources-color-intensity>

PBR

https://www.youtube.com/watch?v=XK_p2MxGBQs

<https://learnopengl.com/PBR/Theory>

<https://www.youtube.com/watch?v=RRE-F57fbXw>

<https://stackoverflow.com/questions/51307278/cook-torrance-shader-with-more-than-one-point-light>

https://www.codinglabs.net/article_physically_based_rendering_cook_torrance.aspx

Este sobretodo (si usé su código)

<https://www.youtube.com/watch?v=gya7x9H3mV0>

Reference code(no usado, solo revisar)

<https://github.com/VictorGordan/opengl-tutorials>

<https://www.rastertek.com/gl4linuxut52.html>

Ray tracing (solo info)

<https://raytracing.github.io/books/RayTracingInOneWeekend.html>

Mapeo de sombras

<https://www.opengl-tutorial.org/intermediate-tutorials/tutorial-16-shadow-mapping/>

Anti aliasing (se siguieron las instrucciones del vídeo)

<https://www.youtube.com/watch?v=oHVh8htoGKw>

Referencia movimiento huesos (anti gravedad)

<https://www.youtube.com/watch?v=mGPlehrzauM>

<https://www.youtube.com/watch?v=1lhxo8GKZQ>

<https://www.youtube.com/watch?v=IjxAZI3-58>

Información

Funcionalidades base:

-Al menos 10 modelos cargados en la pantalla, al menos 3 tipos distintos de VAO

Arco castillo y pared castillo (estructura)
Muros laterales x2
Suelo plane, suelo rocas
Arthur (personaje)
2 calaveras y 3 huesos (movimiento)
Ghost plane (alpha)
Skybox

Total: 13 VAOS, 14 objetos (boneLeg x2)

-Al menos 10 texturas distintas

-Tiene mapas de relieve y mapas especulares o mapas nocturnos/diurnos

-Tiene Mezcla Alfa para al menos una textura semitransparente con mapa alfa

Info general:

-Mapas de AO(hendiduras), baseColor(color), normal(detalle extra), roughness(brillo) cargados para todos los objetos para buen PBR y un look más pulido.
-Fuzz, cavity, bump, gloss, specular displacement disponibles pero no utilizadas debido a falta de tiempo (no eran necesarias)

Info maps utilizados:

-Base Color = Mapa con color base
-Roughness = Mapa alpha que indica qué partes deben tener más brillo o menos. Utilizada en vez de specular por funcionar mejor en este tipo de texturas. Necesario para Cook-Torrance.
-Norma l= uso de rgb para que cada pixel tenga una normal, necesario para la fórmula porque debe tenerse en cuenta de cara al cálculo de luces. Utilizadas en vez de bump por ser más avanzadas que bump y ofrecer 3 direcciones en vez de solo 1. Implementada en `perturbNormal()`.

-Ao = cuánta luz llega dependiendo de la profundidad de las hendiduras, mapa 1-0 alpha. Necesario para Cook-Torrance.

Total:

11 texturas PBR (43 mapas) (Arthur no tiene AO)

1 skydome (simple)

1 textura con alpha (ghost plane)

45 mapas

-Aparece Skybox, sigue a la cámara, la imagen es clara, uso adecuado de Culling y prueba de profundidad

Se activa el culling delantero para poder ver el interior de la esfera (normalmente usamos el back para ver solo el de fuera), y se desactiva el depth test para que esté detrás de los otros objetos. (Debe ser lo primero en dibujarse)

`model_matrix_sky = translate(mat4(1.0f), cameraPos);` Movemos su matriz (identidad) a la posición de la cámara para que siempre la siga.

Antes de dibujar el resto de objetos se vuelve a activar el depth test y el culling trasero para que se dibujen correctamente.

-Se implementó correctamente la cámara en primera persona.

Se implementó un modo FPSEn modo FPS, la matriz de vista se calcula con:

`view_matrix = glm::lookAt(cameraPos, cameraCenter, cameraUp);`

- `cameraPos` es la posición de la cámara
- `cameraCenter` es el punto hacia donde mira
- `cameraUp` define el “arriba” de la cámara.

-Usa Blinn-Phong y la luz viene de la dirección correcta. Utiliza matriz normal

Implementado en la práctica anterior.

La práctica incluye iluminación basada en Blinn-Phong (en el fragment shader).

Blinn-Phong calcula:

- Ambient: una base de luz constante
- Diffuse (Lambert): depende del ángulo entre la normal y la dirección a la luz.
- Specular: brillo especular usando el vector mitad (H), que es la diferencia principal frente a Phong clásico.

En términos de vectores:

- Posición del fragmento: v_vertex
- Normal del fragmento: N
- Vector hacia la cámara: $E = \text{normalize}(u_cam_pos - v_vertex)$
- Vector hacia la luz: $L = \text{normalize}(u_light_dir - v_vertex)$
- Vector mitad: $H = \text{normalize}(L + E)$

L se calcula desde el punto del fragmento hacia la posición/dirección de la luz, lo que hace que el sombreado responda correctamente al mover cámara u objetos.

En el shader, además de transformar los vértices con u_model , también es necesario transformar las normales (los vectores perpendiculares a la superficie que usa la iluminación para calcular difuso y especular).

Si se transformaran usando directamente u_model , cuando el modelo tiene escalado no uniforme (por ejemplo, escalar X distinto a Y) la normal puede quedar “deformada” y dejar de ser realmente perpendicular a la superficie, provocando que la luz y los brillos se vean incorrectos.

De esta forma, las normales quedan coherentes con la geometría transformada y la iluminación, Blinn-Phong/PBR funciona correctamente.

Trayectoria de movimiento complicada con movimiento relativo para al menos 5 modelos

Nuestros 5 objetos en movimiento son 3 huesos y dos calaveras. Estos giran alrededor de un pivote central con un movimiento de ola aproximadamente.

```
vec3 floatingCenter = vec3(-1.0f, -2.7f, 0.0f); (situado entre el arco y Arthur)
```

Primero se definen variables para controlar velocidad, radio interno y externo, altura del movimiento y amplitud de movimiento. (orbitSpeed se calcula multiplicando el valor t, tiempo, por la velocidad que queremos)

Explicación simple del proceso de movimiento.

```
float angleOffset = 0.0f; //Posición inicial del objeto (cada objeto se coloca con un offset con respecto al resto para que no se solapen)
```

```
float angle = orbitSpeed + angleOffset; //Cálculo del ángulo de rotación teniendo en cuenta su velocidad y posición inicial
```

```
//Cálculo de la trayectoria de rotación base de cada objeto dependiendo de su radio y ángulo.
```

```
float x = cos(angle) * outerRadius;  
float z = sin(angle) * outerRadius;
```

```
//Fórmula para calcular movimiento de up down del objeto (uso de los parámetros para crear un tambaleo constante, angle va cambiando, que hace que el objeto se mueva constantemente hacia arriba y abajo imitando un efecto de ola antigravedad)
```

```
float y = baseHeight + sin(angle) * waveHeight;
```

```
//aplicar movimiento
```

```
boneLeg1 = translate(mat4(1.0f), floatingCenter + vec3(x, y, z));
```

```
//Rotación extra sobre su mismo pivote
```

```
boneLeg1 = boneLeg1 * rotate(mat4(1.0f), t * 0.4f, vec3(0, 1, 0));
```

-Por otro lado, como cada objeto tiene su cálculo de movimiento, las calaveras utilizan la variable de inner radius para reducir su recorrido.

Controles de escena

P-Persp-ortho

O-orbital

Up-Down-Horizontal rotation

Left-Right-Vertical rotation

F-FPS

W-Front

S-Back

A-Left-

D-Right

Mouse-Move camera

-Estructura del proyecto

-El programa está bien estructurado utilizando funciones y un uso adecuado de los tipos de datos como estructuras, matrices o clases. Bien comentado y legible. Los nombres de las variables son descriptivos

Resource Files (Shaders):

- `shader.vert` / `shader.frag`: **shader principal** (render de la escena con iluminación/PBR + soporte de sombras mediante `u_lightSpace` y `u_shadowMap`).
- `shader_sky.vert` / `shader_sky.frag`: **shader del sky** (skydome) y se reutiliza para el plano del “ghost” con alpha blending.
- `shadow.vert` / `shadow.frag`: **shader depth-only** para generar el shadow map (calcula `gl_Position`)

Source Files (C++):

- `main.cpp`
Punto de entrada. Se encarga de:
 - inicializar GLFW/GLEW
 - cargar shaders/recursos (`load()`)
 - realizar el render loop (`draw()`)
 - controlar cámara (orbital/FPS) e inputs
- `Shader.h` / `Shader.cpp`
Clase wrapper para compilar y linkar shaders y exponer el program OpenGL.
- `Scene.h` / `Scene.cpp`
Contiene la escena. Centraliza la carga (`LoadAll`) y mantiene los objetos `ObjectPBR`
- `ObjectPBR.h` / `ObjectPBR.cpp`
Representa un objeto renderizable: VAO/índices, texturas/material, y

método Draw(model, view, projection).

- glfunctions.h / glfunctions.cpp
- ObjImp.cpp
- TextureUtils.h / TextureUtils.cpp
Utilidades de texturas
- Librerías en headers:
 - tiny_obj_loader.h
 - stb_image.h: carga de imágenes para texturas (usada en LoadTexture2D()).

Funcionalidades extra:

Múltiples luces

Info:

4 luces (2 main y 2 fill)

Main arco - ilumina el arco desde abajo(rojizo)

Main scene - Ilumina todo el environment(rojizo)

Arthur fill - leve fill en la espalda de Arthur para reducir contraste (azul claro)

Bones fill - fill huesos (muy baja para no sobre iluminar el arco) (azul)

Proceso:

Main

-Defino las variables (dirección-pos, color, intensidad)

```
// Light01 - Main light arco
```

```
vec3 g_light_dir4(-2.5f, 0.0f, -17.5f);
```

```
vec3 g_light_color4(0.7f, 0.1f, 0.15f);
```

```
float g_light_intensity4 = 2.5f;
```

Frag

-Defino los uniform en el frag u_diffuse y u_light_dir.

-Añado la luz al cálculo de Cook Torrance para calcular cuánta luz llega a cada pixel. Y la sumo al cálculo final.

Main

-Uso en el shader

```
//LIGHTS
//01
glUniform3f(glGetUniformLocation(g_simpleShader, "u_light_dir"),
    g_light_dir.x, g_light_dir.y, g_light_dir.z);
glUniform3f(glGetUniformLocation(g_simpleShader, "u_diffuse"),
    g_light_color.x * g_light_intensity,
    g_light_color.y * g_light_intensity,
    g_light_color.z * g_light_intensity);
```

Anti aliasing

Info:

Suavizar los bordes de la geo para evitar bordes de sierra.

Proceso:

-Defino la global unsigned int samples = 8; (n de samples que quiero)

Main

glfwWindowHint(GLFW_SAMPLES, samples); (framebuffer con x samples)

Draw

glEnable(GL_MULTISAMPLE); (activo el multisample)

Se utilizaron las indicaciones de este video informativo.

<https://www.youtube.com/watch?v=oHVh8htoGKw>

Renderizado basado en la física (PBR)

Info:

-Utilizamos el método Cook Torrance para calcular cómo la luz se refleja en las diferentes superficies de nuestra escena.

-Refleja la luz de manera más realista que Blinn-Phong, porque hace uso de microfacetas para saber cómo refleja la luz en cada una de ellas, lo que resulta en un cálculo más preciso en el reflejo de luz desde diferentes ángulos y con diferentes materiales.

Utilizamos este vídeo y su código aplicado a nuestra escena para poder entender mejor todo el concepto de texturas Physically based Rendering.

(Adaptamos el código)

<https://www.youtube.com/watch?v=gya7x9H3mV0>

Proceso:

Funciones:

-fresnelSchlick

- Cálculo del reflejo de luz dependiendo del ángulo. Cuando la vista es lateral refleja más (depende un poco del material, se ve más en metales y demás materiales brillantes)
- Utiliza una fórmula $(F_0 + (1.0 - F_0) * \text{pow}(1.0 - \cos\Theta))$ que calcula el reflejo mediante el ángulo de visión, el reflejo base, y el reflejo calculado a partir del ángulo de visión.

-D_GGX

- Función que calcula según los mapas de roughness cuántas de esas microfacetas están dirigidas (normals) de manera que la luz refleje en ellas.
- Fórmula de Trowbridge Reitz, distribuye, por así decirlo, la luz que refleja cada microfaceta.

-G1_GGX_Schlick

- Dependiendo de la rugosidad de la superficie calcula cuántas microfacetas quedan tapadas por otras desde una dirección.

-G_Smith

- Usa la función G1_GGX_Schlick para calcular las microfacetas que quedan tapadas desde dos puntos (luz y cámara) y devuelve todas las microfacetas ocultas finales.
- Multiplica ambas para calcular el porcentaje de probabilidad de que se den ambos casos.

-brdfMicrofacet (función completa)

- L-dir luz
- V-dir cámara
- N-normal
- Cálculo de H-vector normalizado, average entre la dir de la luz y la de la cam.
- Cálculo de dot products: NoV- ángulo visión cámara; NoL- ángulo de la luz; NoH-alineación de halfway con normal; VoH.
- $f_0 = \text{mix}(f_0, \text{baseColor}, \text{metallic})$; Fórmula que calcula cuánto refleja cuando el ángulo en Fresnel es 0. En metales mucho, en otros materiales poco.
- También nos da su color en valores RGB con el mix del color f_0 original(neutro), mezclado con el baseColor, y la cantidad de metálico que tiene.
- F-cuanto más bajo de el valor Fresnel más bajo es su reflejo. Depende de si es metálico o no.
- D-dependiendo del valor final de GGX se sabe si es una superficie más rugosa o menos.

- G-cuanto más alto sea el resultado de Smith menos ángulos pierden luz por estar bloqueados o no visibles por la cámara.
- Spec- fórmula que calcula la luz que rebota en la superficie por píxel, basándose en el ángulo de vista, la dirección de las microfacetas (si están dirigidas hacia la cámara) y el bloqueo de luz, todo eso normalizado (para que se mantenga en el rango y no devuelva más luz de la que recibe) (Uso de todas las funciones anteriores)(Cuanto más bajo sea menos luz rebota)
- Dif-Calcula el color que no se refleja, que hace diffuse y le da color al objeto. Elimina esto para metales, porque ninguna luz entra, todo rebota. Calcula el color difuso normalizando con PI la mult entre el color base y el difuso.
- Devuelve la suma de la luz difusa y la especular, multiplicado por el ángulo (si viene recta la luz o no)

Final:

```
-vec3 L1 = normalize(u_light_dir - v_vertex); - cálculo de dirección del pixel a la luz
-vec3 pbr_light1 = brdfMicrofacet(L1, E, N, metallic, roughness, texture_color, u_reflectance);- uso de la función que creamos antes para saber cuánta luz le llega a cada píxel mediante cookTorrance
-pbr_light1 *= u_diffuse; - multiplico por la difusa para que el resultado se adapte al color e intensidad de la luz que estoy aplicando.
-Calculo el ambient utilizando el color de la textura, el color e intensidad de la luz ambiente, y el ao del pixel, para saber cuánta luz le llega.(Ambient ilumina todo, pero se suaviza en las grietas)
-La suma final añade el ambient y el efecto de todas las luces que haya en la escena.
-Transformo el RGB a RGBA (mat4) para pasarlo al framebuffer y poder dibujarlo correctamente.
```

Mapeo de sombras

Hemos añadido el mapeo de sombras como funcionalidad extra.

La idea es renderizar la escena dos veces:

- shadow pass (depth pass): se renderiza la escena desde el punto de vista de la luz
- render pass (final pass): se renderiza la escena normal (PBR) desde la cámara, y en el fragment shader se consulta el shadow map para decidir si un punto está en sombra.

En el load() creamos una textura depth y se configuran parámetros. Se crea un g_shadowFBO y se adjunta la textura, después se deshabilita color output con glDrawBuffer(GL_NONE) y glReadBuffer(GL_NONE) ya que solo nos interesa la profundidad.

En el draw() construimos la matriz de lightSpace:

```

glm::mat4 lightView = glm::lookAt(lightPos, lightTarget, glm::vec3(0, 1, 0));
glm::mat4 lightProj = glm::ortho(-15.0f, 15.0f, -15.0f, 15.0f, 1.0f, 60.0f);
glm::mat4 lightSpace = lightProj * lightView;

```

Se renderizan todos los objetos que deben de proyectar sombras, tanto estáticos como animados. Y activamos el culling a FRONT durante el depth pass.

El vertex shader de sombras (shadow.vert) transforma cada vértice al clip space de la luz.

Después del depth pass se vuelve al framebuffer por defecto y al viewport de pantalla y se renderiza sky/escena con g_simpleShader (PBR).

Antes de dibujar la escena PBR se pasa u_lightSpace al shader principal.

A partir de aquí, el shader principal (en su vertex/fragment) proyecta la posición del fragmento a coordenadas de la luz (lightSpace), calcula su profundidad y la compara con el valor almacenado en g_shadowDepthTex.

En shadowPFC subimos el kernel a 25 para que las sombras tuviesen un borde más suave.

```

for (int x = -2; x <= 2; ++x)
for (int y = -2; y <= 2; ++y)
{
    float pcfDepth = texture(u_shadowMap, projCoords.xy + vec2(x, y) *
texelSize).r;
    shadow += (currentDepth - bias > pcfDepth) ? 1.0 : 0.0;
}
shadow /= 25.0;

```